COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	02/09/2026	**DATE SUBMITTED:**	02/09/2026
NAME:	Villaruel, Prince Benjamin S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

SEATWORK # 7 **Algorithm Analysis: Best-Case and Worst-Case Time Complexity**

1. PROCEDURE OUTPUT

What procedures did you perform?

1. Finding an Element

```
None
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        return i;
    }
}
```

Best-Case Time Complexity: O(1)

Worst-Case Time Complexity: O(n)

Explanation: For the best-case time complexity it is found on the first try, while on the worst case time complexity it is found in the end.

2. Printing All Elements

```
None
for (int i = 0; i < n; i++)
{
    cout << arr[i] << endl;
}
```

Best-Case Time Complexity: O(1)

Worst-Case Time Complexity: O(n)

Explanation: It is the same for both the best and complex time complexity since it prints every single item.

3. Nested Loops

```
None
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        cout << i << " " << j << endl;
    }
}
```

Best-Case Time Complexity: O(n^2)

Worst-Case Time Complexity: O(n^2)

Explanation: It is the same for both the best and complex time complexity since the outer loop run n times x the inner loop runs n times (n x n = n^2)

4. Conditional Processing

```
None
if (n > 100)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i];
    }
}
else
{
    cout << "Small input";
}
```

Best-Case Time Complexity: O(1)

Worst-Case Time Complexity: O(n)

Explanation: For the best-case time complexity, it takes 1 step if n is less than or equal to 100 compared to the worst-case time complexity wherein if n is greater than 100.

5. Fixed Inner Loop

```
None
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < 10; j++)
    {
        cout << i << j;
    }
}
```

Best-Case Time Complexity: O(n)

Worst-Case Time Complexity: O(n)

Explanation: It is the same for both the best and complex time complexity since the outer loop runs n times and the inner loop always runs 10 times.

6. Two Consecutive Loops

```
None
for (int i = 0; i < n; i++)
{
    cout << arr[i];
}

for (int j = 0; j < n; j++)
{
    cout << arr[j] * 2;
}
```

Best-Case Time Complexity: O(n)

Worst-Case Time Complexity: O(n)

Explanation: It is the same for both the best and complex time complexity since it is two same loops running back to back.

7. Dividing by Two

```
None
int x = n;

while (x > 1)
{
    x = x / 2;
}
```

Best-Case Time Complexity: O(logn)

Worst-Case Time Complexity: O(logn)

Explanation: It is the same for both the best and complex time complexity since n takes the logarithmic steps to reach 1.

8. Nested Loop With Different Limits

```
None
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < i; j++)
    {
        cout << i << j;
    }
}
```

Best-Case Time Complexity: O(n^2)

Worst-Case Time Complexity: O(n^2)

Explanation: It is the same for both the best and complex time complexity since it runs an average of half n times for each step.

9. Early Termination

```
None
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        cout << "Found";
        break;
    }
}
```

Best-Case Time Complexity: O(1)

Worst-Case Time Complexity: O(n)

Explanation: For the best-case time complexity, it immediately breaks if target is first. While, the worst-case time complexity scans all n items if missing.

10. Two-Level Search

```
None
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (arr[i] == target)
        {
            break;
        }
    }
}
```

Best-Case Time Complexity: O(1)

Worst-Case Time Complexity: O(n^2)

Explanation: For the best-case time complexity, it breaks after the first check. While, the worst-case time complexity checks the entire grid.

11. Linear Loop With Logarithmic Inner Work

```
None
for (int i = 0; i < n; i++)
{
    int x = n;

    while (x > 1)
    {
        x = x / 2;
    }
}
```

Best-Case Time Complexity: O(n log n)

Worst-Case Time Complexity: O(n log n)

Explanation: It is the same for both the best and complex time complexity because the outer loop runs n times and the inner loop halves n(log n) every turn.

12. Conditional Nested Loop

```
None
if (arr[0] == target)
{
    cout << "Found";
}
else
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << arr[i] + arr[j];
        }
    }
}
```

Best-Case Time Complexity: O(1)

Worst-Case Time Complexity: O(n)

Explanation: For the best-case time complexity it matches in the first check. While, the worst-case time complexity it runs the nested loop.

13. Finding the Maximum

```
None
int max = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

Best-Case Time Complexity: O(n)

Worst-Case Time Complexity: O(n)

Explanation: It is the same for both the best and complex time complexity since it must check every element if it guarantees the max.

14. Three Nested Loops

```
None
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        for (int k = 0; k < n; k++)
        {
            cout << i + j + k;
        }
    }
}
```

Best-Case Time Complexity: O(n^3)

Worst-Case Time Complexity: O(n^3)

Explanation: It is the same for both the best and complex time complexity it is three full levels of nested loops (n x n x n = n^3)

15. Comprehensive Analysis

```
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        return i;
    }

    int x = n;

    while (x > 1)
    {
        x = x / 2;
    }
}
```

Best-Case Time Complexity: O(1)

Worst-Case Time Complexity: O(n log n)

Explanation: For the best-case time complexity it immediately returns on the first check. On the other hand, the worst-case time complexity runs n outer loops with log n inner loops each.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.